Investigating Neutrino Oscillations Through Numerical Simulation

.

1. Abstract and Introduction

This project builds a transparent, reproducible Python model of neutrino flavour oscillations, from the two-flavour vacuum approximation to the full three-flavour PMNS framework, and checks every result against an analytical prediction or a physical consistency condition. The numerical first oscillation maximum agrees with theory to 0.015%, the PMNS matrix is unitary to about 10^-16, and neutrino and antineutrino appearance probabilities show the expected CP asymmetry.

Neutrinos are produced and detected as flavour states (electron, muon, tau), but propagate as superpositions of mass eigenstates. Because those components accumulate phase at different rates, a neutrino created with one flavour can later be detected as another. The goal here is not a new physics result, since the phenomenon is well established. It is to understand how the oscillation probability responds to energy, baseline, mixing angles and mass-squared differences, and to build a sound base for more realistic phenomenology.

Objectives

- Implement the two-flavour vacuum oscillation probability and validate it against the analytical oscillation length.
- Isolate how energy, mass-squared difference and mixing angle each shape the oscillation pattern.
- Extend the model to three flavours using the PMNS matrix, with unitarity and probability-conservation checks.
- Quantify the error of the two-flavour approximation and the effect of the CP-violating phase.

Tools: Python, NumPy, SciPy (`find_peaks`), Matplotlib, Jupyter.

2. Theoretical Framework

In the two-flavour approximation, a single mixing angle θ rotates the flavour basis into the mass basis, and the transition probability depends on one mass-squared difference Δm². Flavour states are superpositions of mass eigenstates:

```
\begin{pmatrix} \nu_\alpha \\ \nu_\beta \end{pmatrix} = \begin{pmatrix}
\cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{pmatrix}
\begin{pmatrix} \nu_1 \\ \nu_2 \end{pmatrix}
```

The vacuum transition probability, with L in km, E in GeV and Δm² in eV², is:

```
P(\nu_\alpha \to \nu_\beta) =
\sin^2(2\theta)\,\sin^2\!\left(\frac{1.267\,\Delta m^2 L}{E}\right)
```

The factor sin²(2θ) sets the amplitude, and the phase φ = 1.267 Δm² L / E sets how quickly the pattern develops with distance. Since sin²φ repeats when φ advances by π, one full cycle spans the oscillation length:

```
L_{\mathrm{osc}} = \frac{\pi E}{1.267\,|\Delta m^2|}
```

The first maximum falls at L_osc / 2. This gives three testable predictions: energy lengthens L_osc, Δm² shortens it, and θ changes only the height. Sections 4 and 5 test each of them numerically.

3. Two-Flavour Implementation

The baseline model uses θ = 33°, Δm² = 2.4×10⁻³ eV² and E = 2 GeV over 0–6000 km. The probability is vectorised with NumPy, so every distance is evaluated at once with no loop.

```
import numpy as np
import matplotlib.pyplot as plt

# Physical parameters
theta_deg = 33
theta = np.deg2rad(theta_deg)

delta_m2 = 2.4e-3  # Mass-squared difference in eV^2
energy = 2.0       # Neutrino energy in GeV

# Propagation distance in kilometres
L = np.linspace(0, 6000, 10000)
```

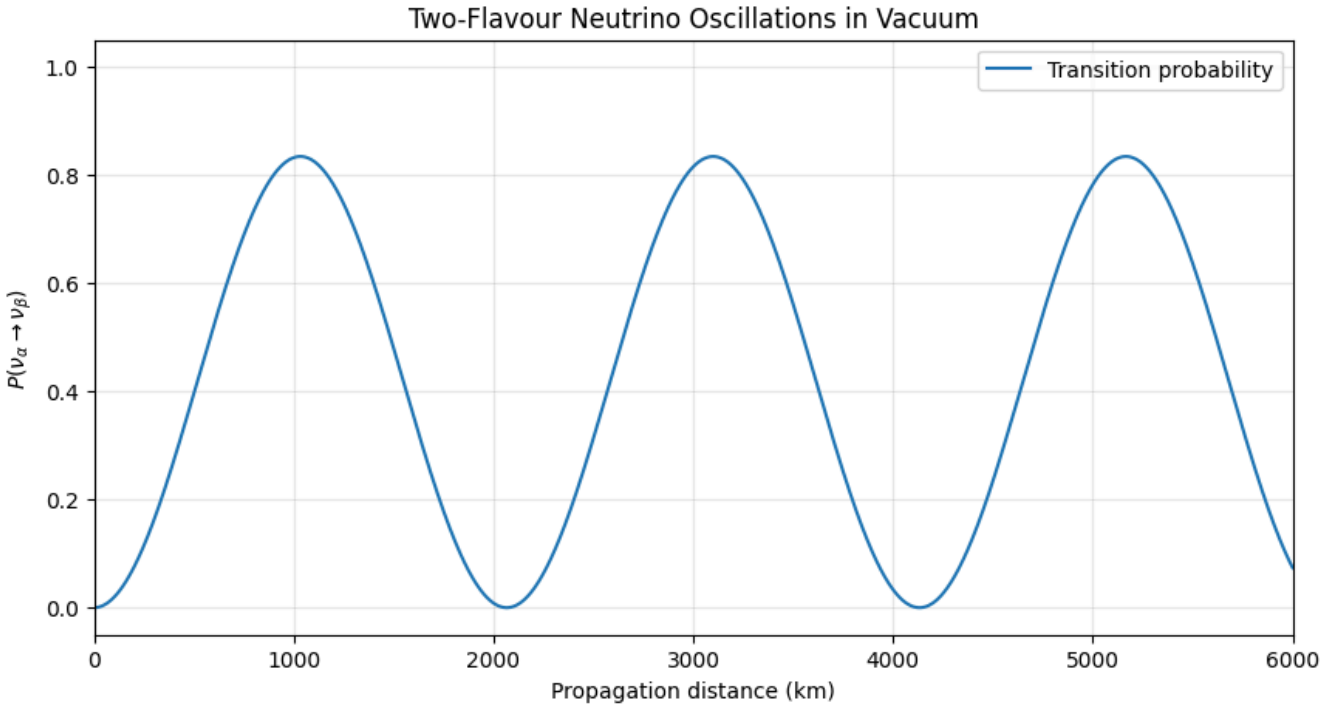
```
# Calculate the transition probability
P_transition = (
    np.sin(2 * theta)**2
    * np.sin(1.267 * delta_m2 * L / energy)**2
)

# Basic numerical checks
print(f"Minimum probability: {P_transition.min():.6f}")
print(f"Maximum probability: {P_transition.max():.6f}")
print(f"Probability at zero distance: {P_transition[0]:.6f}")

assert np.all(P_transition >= 0)
assert np.all(P_transition <= 1)
assert np.isclose(P_transition[0], 0)
```

Output: minimum 0.000000, maximum 0.834565, probability at zero distance 0.000000, and all sanity checks passed.

```
plt.figure(figsize=(10, 5))
plt.plot(L, P_transition, label="Transition probability")
plt.xlabel("Propagation distance (km)")
plt.ylabel(r"$P(\nu_\alpha \rightarrow \nu_\beta)$")
plt.title("Two-Flavour Neutrino Oscillations in Vacuum")
plt.ylim(-0.05, 1.05)
plt.xlim(0, 6000)
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```



The probability starts at zero, rises to a maximum, falls back and repeats, rather than growing without limit. The peak is 0.835 rather than 1 because $\sin^2(2\times33°) = 0.835$: only a 45° mixing angle allows a complete transition.

4. Validation Against the Analytical Oscillation Length

The simulation reproduces the analytical oscillation length: the first numerical maximum lands within 0.157 km (0.015%) of the prediction. The check has three steps: compute L_osc analytically, locate maxima and minima numerically with SciPy's `find_peaks` (minima as peaks of the negated array), then compare.

```
# Theoretical oscillation length; first maximum is at half a cycle
L_osc_theory = np.pi * energy / (1.267 * abs(delta_m2))
L_first_max_theory = L_osc_theory / 2

print(f"Theoretical oscillation length: {L_osc_theory:.2f} km")
print(f"Predicted first maximum: {L_first_max_theory:.2f} km")
```

```
from scipy.signal import find_peaks

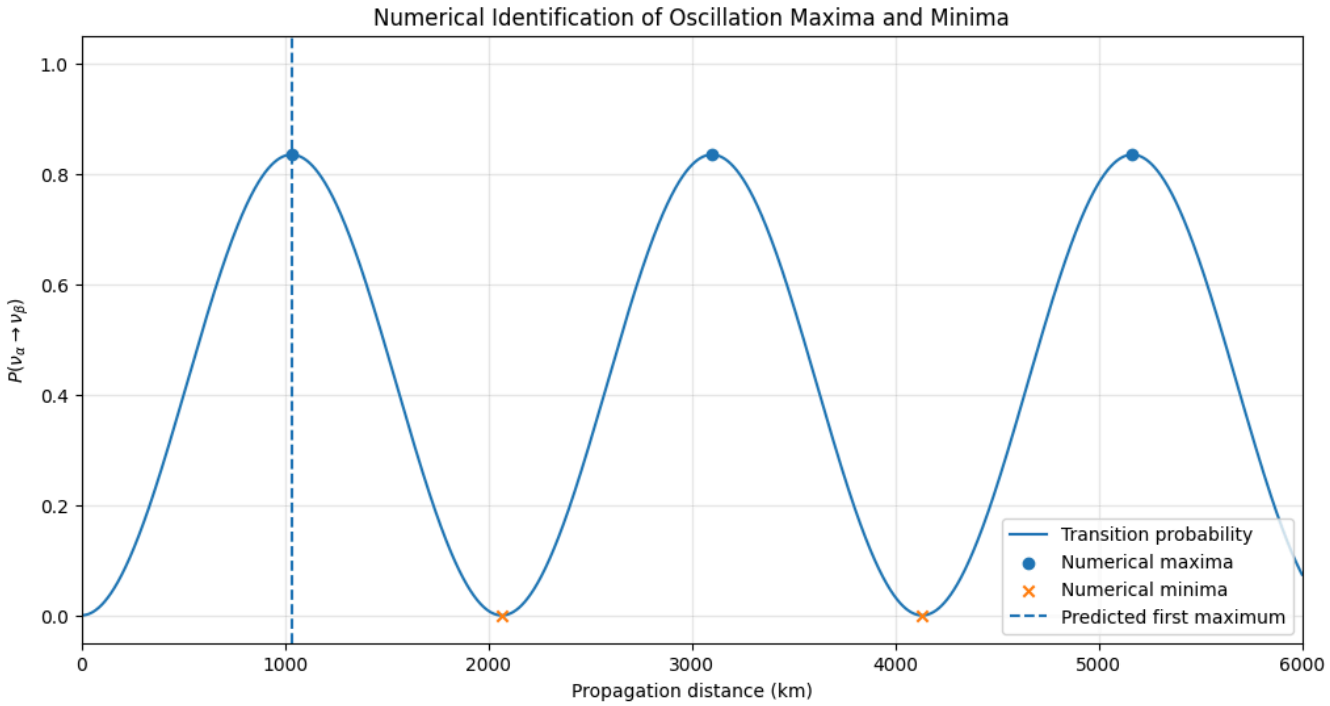
peak_indices, _ = find_peaks(P_transition)         # local maxima
minimum_indices, _ = find_peaks(-P_transition)     # local minima

peak_distances = L[peak_indices]
peak_probabilities = P_transition[peak_indices]
minimum_distances = L[minimum_indices]
minimum_probabilities = P_transition[minimum_indices]

print("First five local maxima:")
for distance, probability in zip(peak_distances[:5], peak_probabilities[:5]):
    print(f"Distance = {distance:.2f} km, Probability = {probability:.6f}")

print("\nFirst five local minima:")
for distance, probability in zip(minimum_distances[:5],
minimum_probabilities[:5]):
    print(f"Distance = {distance:.2f} km, Probability = {probability:.6f}")
```

```
plt.figure(figsize=(12, 6))
plt.plot(L, P_transition, label="Transition probability")
plt.scatter(peak_distances, peak_probabilities, marker="o",
            label="Numerical maxima", zorder=3)
plt.scatter(minimum_distances, minimum_probabilities, marker="x",
            label="Numerical minima", zorder=3)
plt.axvline(L_first_max_theory, linestyle="--", label="Predicted first
maximum")
plt.xlabel("Propagation distance (km)")
plt.ylabel(r"$P(\nu_\alpha \rightarrow \nu_\beta)$")
plt.title("Numerical Identification of Oscillation Maxima and Minima")
plt.xlim(0, 6000); plt.ylim(-0.05, 1.05)
plt.grid(True, alpha=0.3); plt.legend()
plt.show()
```




```
L_first_max_numerical = peak_distances[0]
absolute_difference = abs(L_first_max_numerical - L_first_max_theory)
relative_difference = absolute_difference / L_first_max_theory * 100

print(f"Theoretical first maximum: {L_first_max_theory:.3f} km")
print(f"Numerical first maximum:   {L_first_max_numerical:.3f} km")
print(f"Absolute difference:       {absolute_difference:.3f} km")
print(f"Relative difference:       {relative_difference:.4f}%")
```

Quantity	Theory	Numerical
Oscillation length (km)	2066.29	2066.29 (period)
First maximum (km)	1033.15	1033.30
Second maximum (km)	3099.44	3099.31
Third maximum (km)	5165.74	5165.92
Minima (km)	2066.29, 4132.59	2066.01, 4132.61
Peak probability	0.834565	0.834565

The first-maximum difference is 0.157 km, a relative error of 0.0152%. The residual comes from the finite grid spacing of 0.6 km (10,000 points over 6000 km), not from the physics.

5. Parameter Dependence

Varying one parameter at a time shows that energy and mass-squared difference control the oscillation length, while the mixing angle controls only the amplitude. All three plots share the axis formatting of Figure 1 (0–6000 km, y from −0.05 to 1.05, grid, legend), omitted below for brevity.

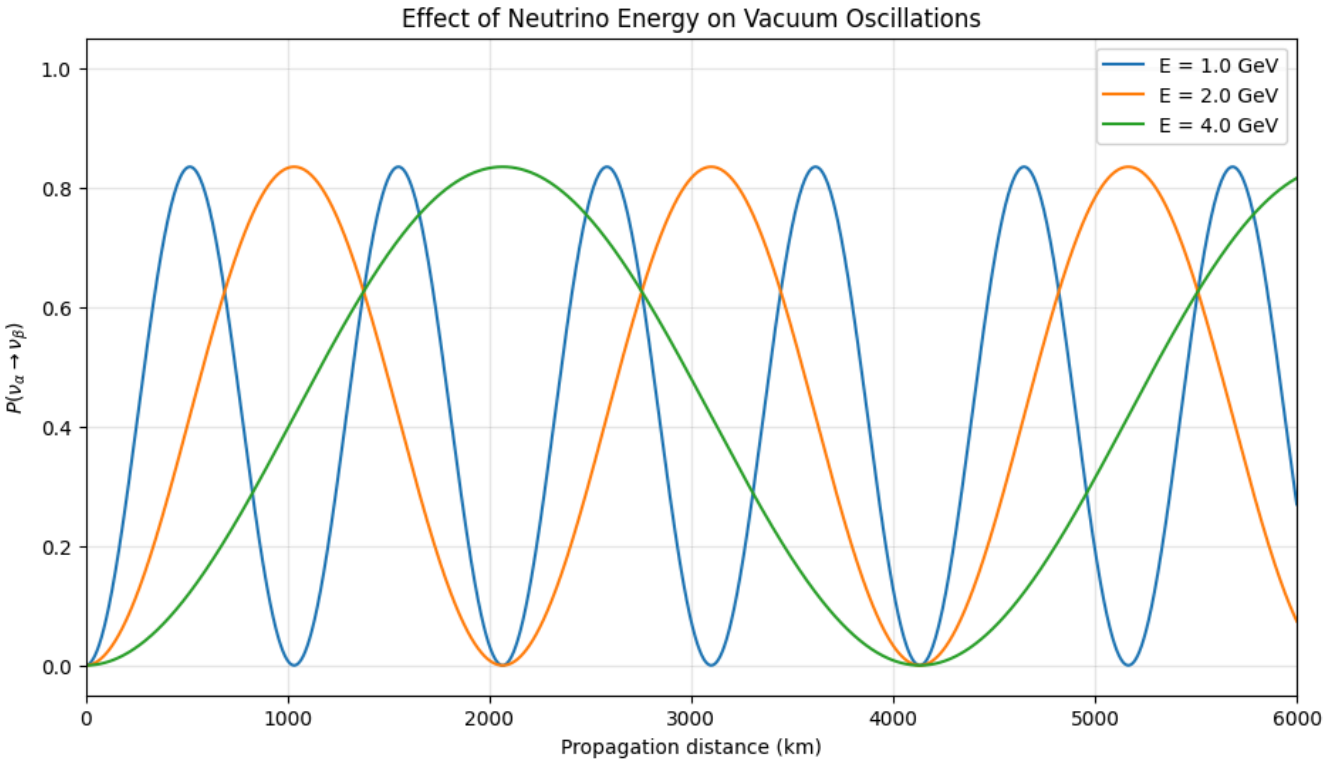
5.1 Neutrino energy

L_osc is proportional to E, so doubling the energy doubles the oscillation length.

```
energies = [1.0, 2.0, 4.0]  # GeV

plt.figure(figsize=(11, 6))
for E in energies:
    P = np.sin(2 * theta)**2 * np.sin(1.267 * delta_m2 * L / E)**2
    plt.plot(L, P, label=f"E = {E:.1f} GeV")

print("Theoretical oscillation lengths")
for E in energies:
    L_osc = np.pi * E / (1.267 * abs(delta_m2))
    print(f"E = {E:.1f} GeV -> L_osc = {L_osc:.2f} km")
```



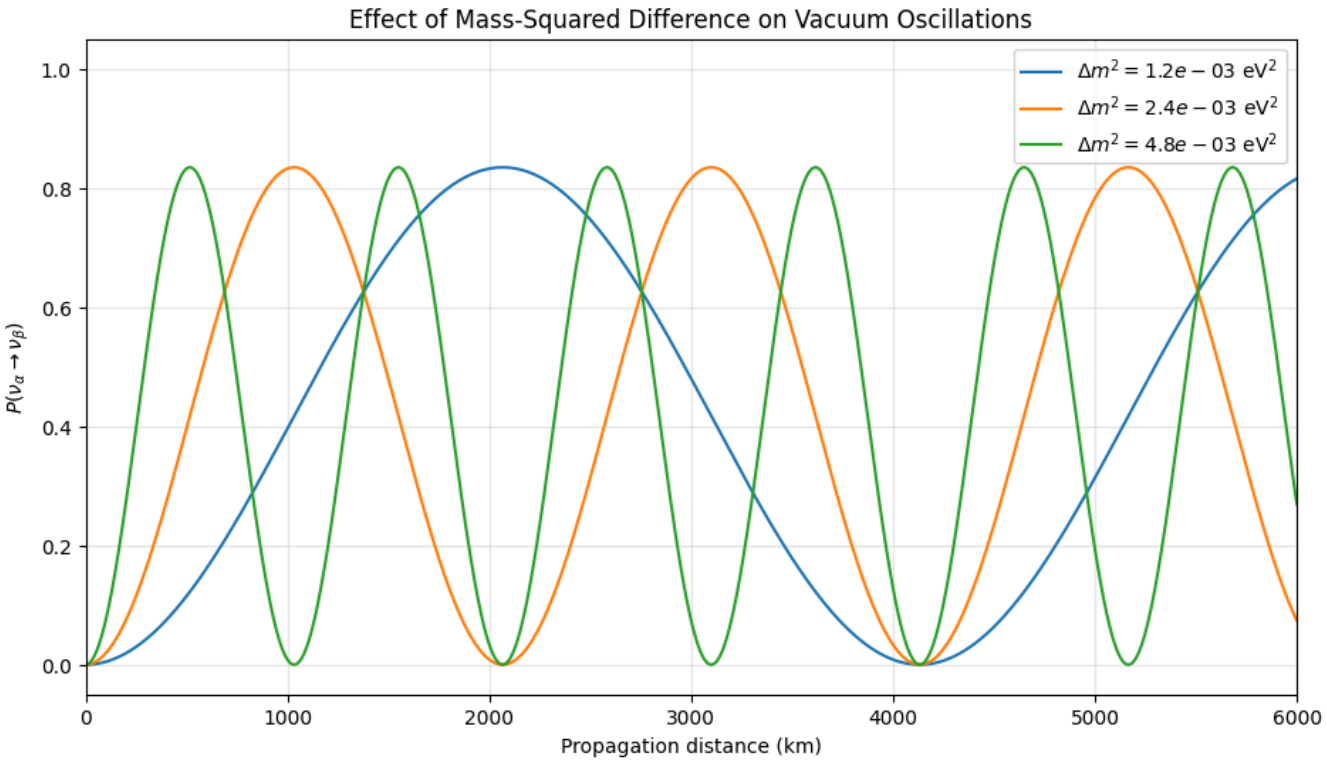
5.2 Mass-squared difference

L_osc is inversely proportional to |Δm²|, so a larger splitting compresses the pattern.

```
mass_squared_differences = [1.2e-3, 2.4e-3, 4.8e-3]  # eV^2

plt.figure(figsize=(11, 6))
for dm2 in mass_squared_differences:
    P = np.sin(2 * theta)**2 * np.sin(1.267 * dm2 * L / energy)**2
    plt.plot(L, P, label=rf"$\Delta m^2 = {dm2:.1e}\ \mathrm{{eV}}^2$")

print("Theoretical oscillation lengths")
for dm2 in mass_squared_differences:
    L_osc = np.pi * energy / (1.267 * abs(dm2))
    print(f"Delta m^2 = {dm2:.1e} eV^2 -> L_osc = {L_osc:.2f} km")
```

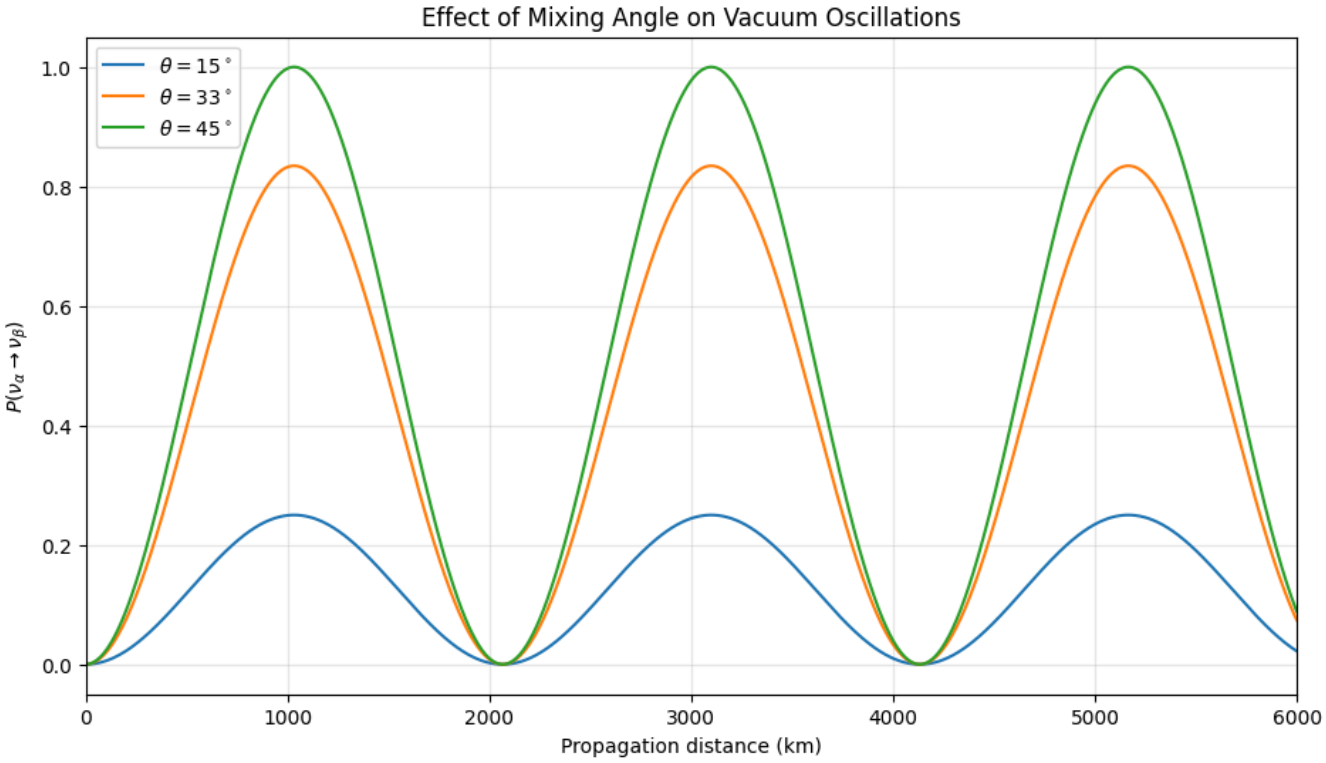


5.3 Mixing angle

The amplitude sin²(2θ) changes with the angle, but the spacing of maxima does not.

```
mixing_angles_deg = [15, 33, 45]

plt.figure(figsize=(11, 6))
for angle_deg in mixing_angles_deg:
    angle = np.deg2rad(angle_deg)
    P = np.sin(2 * angle)**2 * np.sin(1.267 * delta_m2 * L / energy)**2
    plt.plot(L, P, label=rf"$\theta = {angle_deg}^\circ$")
```



5.4 Summary

Parameter	Values tested	Oscillation length (km)	Maximum probability
Energy E	1, 2, 4 GeV	1033.15, 2066.29, 4132.59	unchanged (0.835)
Mass-squared difference Δm^2	1.2, 2.4, 4.8 × 10^{-3} eV^2	4132.59, 2066.29, 1033.15	unchanged (0.835)
Mixing angle θ	15°, 33°, 45°	unchanged (2066.29)	$\sin^2(2\theta)$ = 0.25, 0.835, 1.0

The probability depends on L/E, so a lower energy and a larger Δm^2 are equivalent in their effect on the oscillation pattern.

6. Three-Flavour Oscillations

The three-flavour model is built on the PMNS matrix and passes both consistency tests: $U^\dagger U$ equals the identity to numerical precision, and the flavour probabilities sum to exactly 1 at every distance tested.

6.1 The PMNS matrix

Flavour states are linked to the mass eigenstates ν_1, ν_2, ν_3 through three mixing angles and a Dirac CP phase δ (with s = sin and c = cos):

```
U = \begin{pmatrix} c_{12}c_{13} & s_{12}c_{13} & s_{13}e^{-i\delta} \\ -
s_{12}c_{23}-c_{12}s_{23}s_{13}e^{i\delta} & c_{12}c_{23}-
s_{12}s_{23}s_{13}e^{i\delta} & s_{23}c_{13} \\ s_{12}s_{23}-
c_{12}c_{23}s_{13}e^{i\delta} & -c_{12}s_{23}-s_{12}c_{23}s_{13}e^{i\delta} &
c_{23}c_{13} \end{pmatrix}
```

Parameter	Value
θ_{12}	33.4°
θ_{23}	49.0°
θ_{13}	8.6°
δ (reference)	195°
Δm^2_{21}	7.42×10^{-5} eV²
Δm^2_{31}	2.517×10^{-3} eV²

```
import numpy as np

theta12_deg, theta23_deg, theta13_deg = 33.4, 49.0, 8.6
theta12 = np.deg2rad(theta12_deg)
theta23 = np.deg2rad(theta23_deg)
theta13 = np.deg2rad(theta13_deg)

delta_cp_deg = 195.0
```

```
delta_cp = np.deg2rad(delta_cp_deg)

s12, c12 = np.sin(theta12), np.cos(theta12)
s23, c23 = np.sin(theta23), np.cos(theta23)
s13, c13 = np.sin(theta13), np.cos(theta13)

U = np.array([
    [c12 * c13, s12 * c13, s13 * np.exp(-1j * delta_cp)],
    [-s12 * c23 - c12 * s23 * s13 * np.exp(1j * delta_cp),
      c12 * c23 - s12 * s23 * s13 * np.exp(1j * delta_cp),
      s23 * c13],
    [ s12 * s23 - c12 * c23 * s13 * np.exp(1j * delta_cp),
     -c12 * s23 - s12 * c23 * s13 * np.exp(1j * delta_cp),
      c23 * c13]
], dtype=complex)

unitarity_check = U.conj().T @ U
assert np.allclose(unitarity_check, np.eye(3), atol=1e-10)
print("Unitarity check passed.")
```

The resulting matrix for δ = 195° is:

```
[[ 0.8255+0.j      0.5443+0.j     -0.1444+0.0387j]
 [-0.2701+0.0244j  0.6077+0.0161j  0.7462+0.j    ]
 [ 0.4946+0.0212j -0.5779+0.014j   0.6487+0.j    ]]
```

6.2 Propagation in vacuum

Each mass eigenstate acquires a phase, and the transition amplitude is a coherent sum over the three states:

```
P_{\alpha\to\beta} = \left| \sum_{i=1}^{3} U_{\beta i}\,U_{\alpha i}^{*}\,
e^{-i m_i^2 L / (2E)} \right|^2
```

An overall common phase drops out of the probability, so oscillations depend only on mass-squared differences. The code therefore sets $m_1^2 = 0$, a computational convenience rather than a statement about the lightest neutrino mass. The factor 2×1.267 converts $m^2L/2E$ to practical units (km, GeV, eV²).

```
delta_m21_sq = 7.42e-5  # eV^2
delta_m31_sq = 2.517e-3 # eV^2
mass_sq = np.array([0.0, delta_m21_sq, delta_m31_sq])

def three_flavour_probability(alpha, beta, distance_km, energy_GeV):
    """Vacuum probability alpha -> beta. 0 = nu_e, 1 = nu_mu, 2 = nu_tau."""
    phase = 2 * 1.267 * mass_sq * distance_km / energy_GeV
    amplitude = np.sum(U[beta, :] * np.conj(U[alpha, :]) * np.exp(-1j *
phase))
    return np.abs(amplitude)**2
```

At 1000 km and 2 GeV, an electron neutrino has these outcomes:

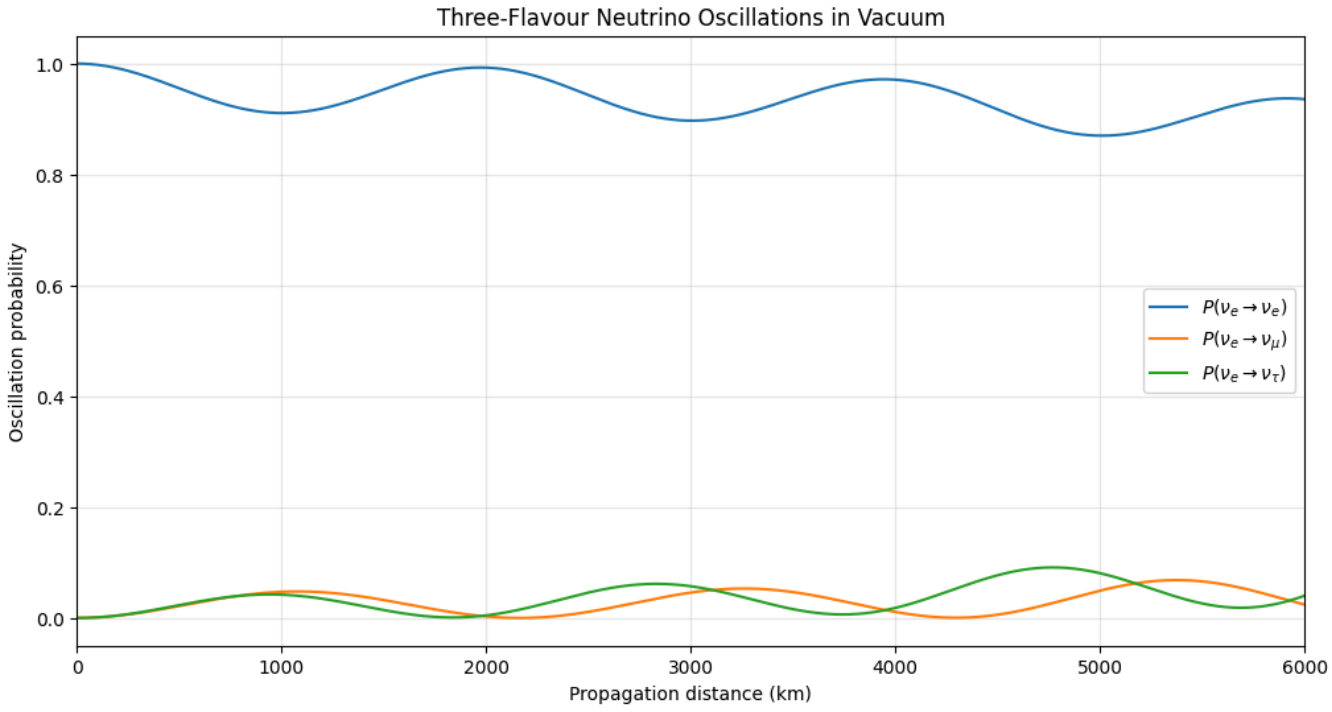
Channel	Probability
$\nu_e \to \nu_e$	0.910823
$\nu_e \to \nu\mu$	0.047197
$\nu_e \to \nu\tau$	0.041981
Sum	1.000000

6.3 Probabilities over distance

```
energy_3f = 2.0  # GeV
P_ee   = np.array([three_flavour_probability(0, 0, d, energy_3f) for d in L])
P_emu  = np.array([three_flavour_probability(0, 1, d, energy_3f) for d in L])
P_etau = np.array([three_flavour_probability(0, 2, d, energy_3f) for d in L])

probability_sum = P_ee + P_emu + P_etau
assert np.allclose(probability_sum, 1.0, atol=1e-10)

plt.figure(figsize=(12, 6))
plt.plot(L, P_ee,   label=r"$P(\nu_e \rightarrow \nu_e)$")
plt.plot(L, P_emu,  label=r"$P(\nu_e \rightarrow \nu_\mu)$")
plt.plot(L, P_etau, label=r"$P(\nu_e \rightarrow \nu_\tau)$")
plt.xlabel("Propagation distance (km)")
plt.ylabel("Oscillation probability")
plt.title("Three-Flavour Neutrino Oscillations in Vacuum")
plt.xlim(0, 6000); plt.ylim(-0.05, 1.05)
plt.grid(True, alpha=0.3); plt.legend()
plt.show()
```



The electron-neutrino survival probability stays above about 0.87: it dips as probability moves into the muon and tau channels, then recovers as the mass-eigenstate contributions interfere. This is a small-amplitude pattern, unlike the near-complete

transitions of the two-flavour toy model, because θ_{13} is small. The sum stays at 1.0000000000 across the whole range, which confirms the implementation conserves probability.

7. Two-Flavour Approximation versus the Full Model

At 1000 km and 2 GeV the two-flavour approximation for $\nu\mu \to \nu_e$ is within 0.0039 (about 7%) of the full three-flavour result, but the gap grows with distance. The approximation keeps only the dominant atmospheric term:

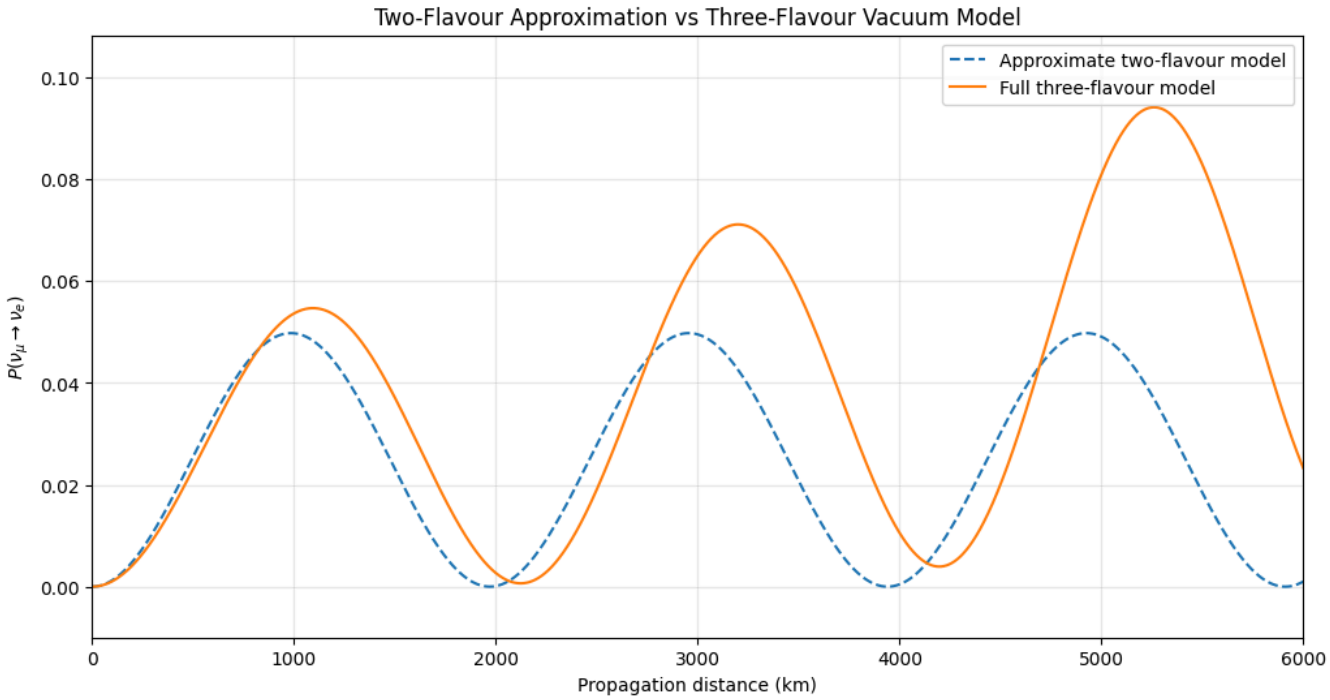
```
P(\nu_\mu \to \nu_e) \approx \sin^2\theta_{23}\,\sin^2
2\theta_{13}\,\sin^2\!\left(\frac{1.267\,\Delta m^2_{31} L}{E}\right)
```

```
energy_comparison = 2.0  # GeV
theta23_rad = np.deg2rad(theta23_deg)
theta13_rad = np.deg2rad(theta13_deg)

P_mue_approx = (
    np.sin(theta23_rad)**2
    * np.sin(2 * theta13_rad)**2
    * np.sin(1.267 * delta_m31_sq * L / energy_comparison)**2
)

P_mue_3f = np.array([
    three_flavour_probability(alpha=1, beta=0, distance_km=d,
                              energy_GeV=energy_comparison)
    for d in L
])

plt.figure(figsize=(12, 6))
plt.plot(L, P_mue_approx, linestyle="--", label="Approximate two-flavour
model")
plt.plot(L, P_mue_3f, label="Full three-flavour model")
plt.xlabel("Propagation distance (km)")
plt.ylabel(r"$P(\nu_\mu \rightarrow \nu_e)$")
plt.title("Two-Flavour Approximation vs Three-Flavour Vacuum Model")
plt.xlim(0, 6000)
plt.ylim(-0.01, max(P_mue_3f.max(), P_mue_approx.max()) * 1.15)
plt.grid(True, alpha=0.3); plt.legend()
plt.show()
```




```
comparison_distance = 1000
P_approx_1000 = (np.sin(theta23_rad)**2 * np.sin(2 * theta13_rad)**2
                 * np.sin(1.267 * delta_m31_sq * comparison_distance
                          / energy_comparison)**2)
P_3f_1000 = three_flavour_probability(1, 0, comparison_distance,
energy_comparison)
```

Quantity (1000 km, 2 GeV)	Value
Two-flavour approximation	0.049778
Three-flavour model	0.053666
Absolute difference	0.003888

The approximation tracks the first maximum but fails to reproduce the slower modulation of the full model, where successive peaks grow instead of staying at a constant height. The full model additionally contains the solar splitting Δm^2_{21} and CP-phase-dependent interference. The two-flavour model remains a good guide to the basic physics, but it cannot capture every feature. Both calculations assume vacuum propagation, and the comparison is numerical, not a validation against experimental data.

8. The CP-Violating Phase

The model shows CP violation as designed: the neutrino and antineutrino probabilities are identical for δ = 0° and 180°, and differ by ±0.025 at 1000 km for δ = 90° and 270°. Antineutrino probabilities are computed by replacing U with its complex conjugate, and the asymmetry is defined as:

```
A_{CP} = P(\nu_\mu \to \nu_e) - P(\bar{\nu}_\mu \to \bar{\nu}_e)
```

The PMNS construction is wrapped in a function so δ can be varied. Every matrix is checked for unitarity before use.

```
def build_pmns_matrix(theta12_deg, theta23_deg, theta13_deg, delta_cp_deg):
    t12, t23, t13 = np.deg2rad([theta12_deg, theta23_deg, theta13_deg])
    delta = np.deg2rad(delta_cp_deg)
    s12, c12 = np.sin(t12), np.cos(t12)
    s23, c23 = np.sin(t23), np.cos(t23)
    s13, c13 = np.sin(t13), np.cos(t13)
    exp_plus, exp_minus = np.exp(1j * delta), np.exp(-1j * delta)

    return np.array([
        [c12*c13, s12*c13, s13*exp_minus],
        [-s12*c23 - c12*s23*s13*exp_plus,
          c12*c23 - s12*s23*s13*exp_plus, s23*c13],
        [ s12*s23 - c12*c23*s13*exp_plus,
         -c12*s23 - s12*c23*s13*exp_plus, c23*c13]
    ], dtype=complex)

delta_values = [0, 90, 180, 270, 195]
pmns_matrices = {d: build_pmns_matrix(33.4, 49.0, 8.6, d) for d in
delta_values}

for delta, U_test in pmns_matrices.items():
    dev = np.max(np.abs(U_test.conj().T @ U_test - np.eye(3)))
    print(f"delta_CP = {delta:3d} deg | max unitarity deviation: {dev:.2e}")
```

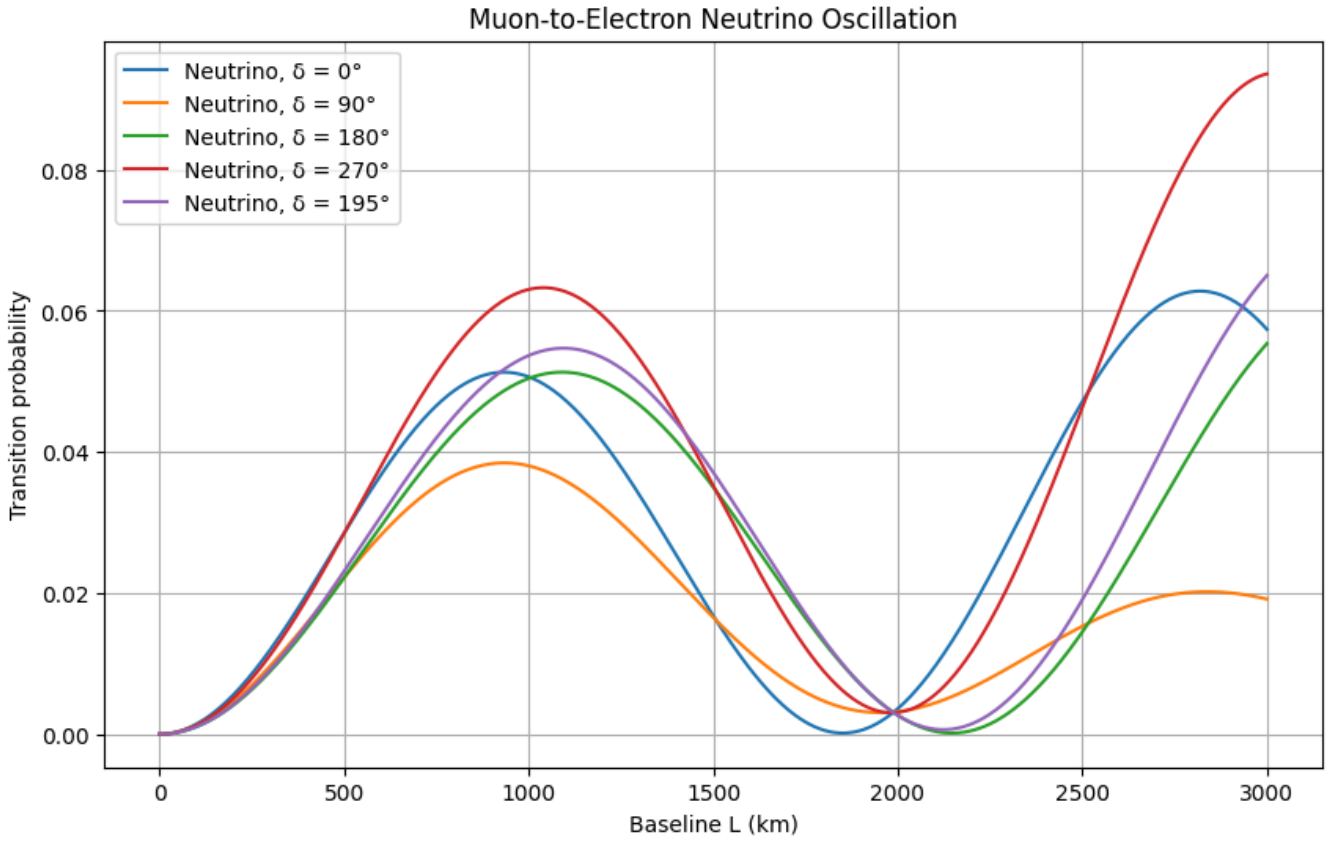
The maximum unitarity deviation is at most 2.2×10^{-16} for all five phases (6.27e-17, 2.95e-17, 5.65e-17, 4.44e-17 and 2.22e-16 for δ = 0°, 90°, 180°, 270°, 195°).

```
def oscillation_probability(alpha, beta, distance_km, energy_GeV, U_matrix):
    phase = 2 * 1.267 * mass_sq * distance_km / energy_GeV
    amplitude = np.sum(U_matrix[beta, :] * np.conj(U_matrix[alpha, :])
                       * np.exp(-1j * phase))
    return np.abs(amplitude)**2

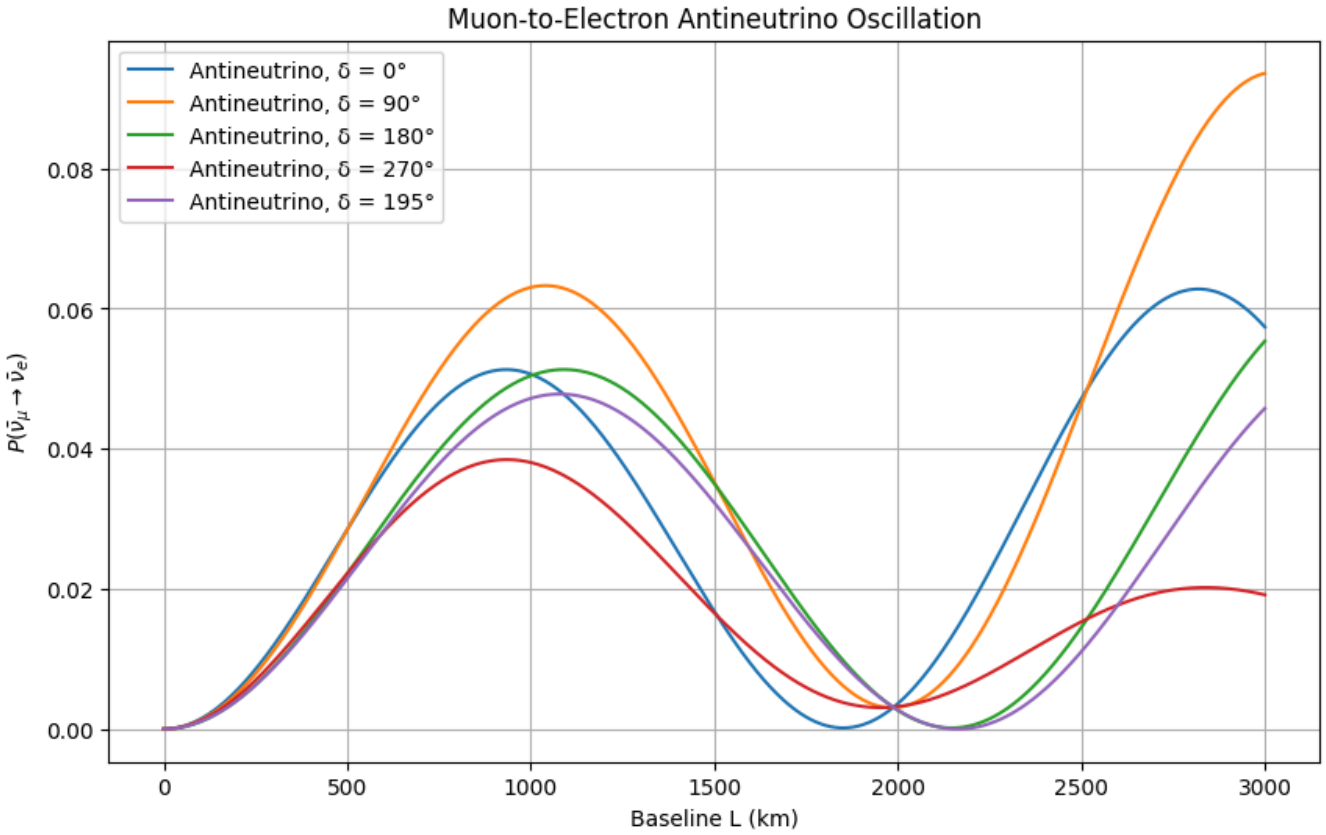
energy_cp = 2.0
L_cp = np.linspace(0, 3000, 3000)
delta_to_plot = [0, 90, 180, 270, 195]
```

8.1 Neutrino and antineutrino probabilities

```
# Neutrinos: use U
plt.figure(figsize=(10, 6))
for delta in delta_to_plot:
    U_test = pmns_matrices[delta]
    P_nu = np.array([oscillation_probability(1, 0, d, energy_cp, U_test)
                     for d in L_cp])
    plt.plot(L_cp, P_nu, label=f"Neutrino, delta = {delta} deg")
plt.xlabel("Baseline L (km)")
plt.ylabel("Transition probability")
plt.title("Muon-to-Electron Neutrino Oscillation")
plt.legend(); plt.grid(True); plt.show()
```



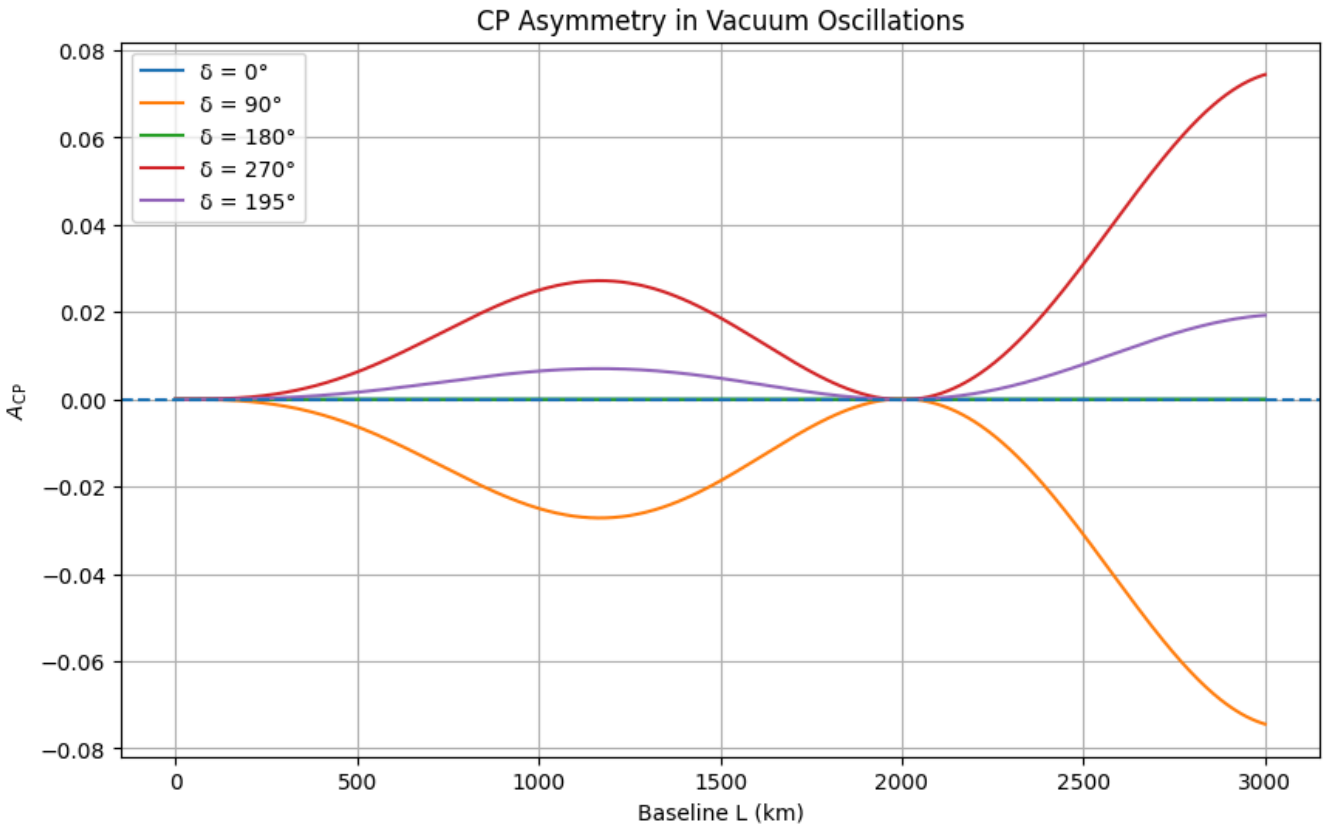

```
# Antineutrinos: use the complex conjugate of U
plt.figure(figsize=(10, 6))
for delta in delta_to_plot:
    U_test = pmns_matrices[delta]
    P_anti = np.array([oscillation_probability(1, 0, d, energy_cp,
np.conj(U_test))
                       for d in L_cp])
    plt.plot(L_cp, P_anti, label=f"Antineutrino, delta = {delta} deg")
plt.xlabel("Baseline L (km)")
plt.ylabel(r"$P(\bar{\nu}_\mu \rightarrow \bar{\nu}_e)$")
plt.title("Muon-to-Electron Antineutrino Oscillation")
plt.legend(); plt.grid(True); plt.show()
```



8.2 CP asymmetry

```
plt.figure(figsize=(10, 6))
for delta in delta_to_plot:
    U_test = pmns_matrices[delta]
    P_nu   = np.array([oscillation_probability(1, 0, d, energy_cp, U_test)
                       for d in L_cp])
    P_anti = np.array([oscillation_probability(1, 0, d, energy_cp,
np.conj(U_test))
                       for d in L_cp])
    plt.plot(L_cp, P_nu - P_anti, label=f"delta = {delta} deg")

plt.axhline(0, linestyle="--")
plt.xlabel("Baseline L (km)")
plt.ylabel(r"$A_{\mathrm{CP}}$")
plt.title("CP Asymmetry in Vacuum Oscillations")
plt.legend(); plt.grid(True); plt.show()
```



The asymmetry is zero at δ = 0° and 180° at every baseline. The 90° and 270° curves are mirror images of each other, and all curves return to zero near 2000 km before growing again. The reference value δ = 195° lies close to 180°, so its asymmetry is small.

8.3 Numerical comparison at 1000 km

```
baseline_test = 1000  # km
for delta in delta_to_plot:
    U_test = pmns_matrices[delta]
    P_nu   = oscillation_probability(1, 0, baseline_test, energy_cp, U_test)
    P_anti = oscillation_probability(1, 0, baseline_test, energy_cp,
np.conj(U_test))
    print(f"delta = {delta:3d} | P = {P_nu:.5f} | Pbar = {P_anti:.5f} | A_CP
= {P_nu - P_anti:+.5f}")
```

δ (deg)	P(νμ → νe)	P(ν̄μ → ν̄e)	A_CP
0	0.05065	0.05065	+0.00000
90	0.03804	0.06304	−0.02500

δ (deg)	$P(\nu\mu \to \nu_e)$	$P(\bar{\nu}\mu \to \bar{\nu}_e)$	A_CP
180	0.05043	0.05043	+0.00000
270	0.06304	0.03804	+0.02500
195	0.05367	0.04720	+0.00647

In vacuum, a nonzero asymmetry signals CP violation through the complex phase. In a real experiment, matter effects also differ between neutrinos and antineutrinos, so an observed asymmetry would not by itself prove intrinsic CP violation. This model does not include those effects.

9. Conclusions, Limitations and Future Work

The simulation reproduces the expected physics of neutrino oscillations at each stage, and every result passed an independent check.

Findings

- Two-flavour model: oscillation length and maxima match theory, with the first maximum within 0.015%.
- Parameter dependence: energy and Δm^2 set the oscillation length (L_{osc} proportional to E and to $1/|\Delta m^2|$), while the mixing angle sets the amplitude $\sin^2(2\theta)$.
- Three-flavour model: the PMNS matrix is unitary to about 10^{-16} and probabilities sum to 1 at every baseline.
- Model comparison: the two-flavour approximation differs from the full model by 0.0039 at 1000 km and 2 GeV, and the gap grows with distance.
- CP violation: the asymmetry vanishes for $\delta = 0°$ and 180° and reaches ±0.025 at $\delta = 90°$ and 270° (1000 km, 2 GeV).

Limitations

- Propagation is in vacuum with coherent oscillations; matter effects, which matter for neutrinos crossing the Earth, are not included.
- The results are theoretical predictions from fixed oscillation parameters. They are not fitted to experimental data and set no constraints on neutrino properties.
- The two-flavour model uses illustrative parameters, and the Δm^2 scan values were chosen to show the dependence, not all to be physical.

Future work

- Add constant-density matter effects (MSW) and separate them from intrinsic CP violation.
- Compare against public experimental data from long-baseline experiments.
- Extend the CP study to a scan over δ and energy, and test sensitivity to the mass ordering.

Reproducibility: the full analysis is a single Jupyter notebook using NumPy, SciPy and Matplotlib. All code above runs top to bottom in the order shown.